6. `app/controllers/approved_senders_controller.rb`
7. `app/models/approved_sender.rb`
8. Any directly related `TripItem`, `Expense`, `Trip` or routing code needed to complete the flow.

I want the investigation to answer these questions:

1. How does an incoming email enter the application and get associated with a trip?
2. What does `TripMailbox` itself decide or create?
3. What information is stored in `InboxItem` before parsing starts?
4. Exactly what does `ParseInboxItemJob` do?
5. What responsibility belongs to Anthropic and what responsibility remains in DaPlanStan code?
6. Does parsing itself create `TripItem` or `Expense` records, or does that happen later through confirmation/update logic?
7. Trace the exact conditions and call sites for methods such as `fill_missing_links!` or any equivalent record-linking methods.
8. How does approved-sender logic affect the workflow?
9. Which parts are application-written, which are Rails/Action Mailbox/Active Job behavior, and which are dependency/provider behavior?
10. Record the important source paths and reliable line ranges.

Once the collaboration is fully traced, identify any pattern classifications that are genuinely worth investigating.
For each possible pattern:

1. State the candidate pattern.
2. Explain the specific code behavior that suggests it.
3. Explain the strongest reason it may be a false positive.
4. State whether the apparent behavior is mainly application-owned, Rails-provided or dependency-provided.
5. Leave the candidate UNCONFIRMED in this step.

Do not force patterns such as Observer, Command, Chain of Responsibility, Adapter, Strategy or Factory merely because the workflow contains jobs, messages or multiple steps.
At the end give me:

1. Complete inbound-email collaboration trace.
2. Participant/source-evidence table.
3. Clear separation of application, Rails and Anthropic responsibilities.
4. Candidate pattern classifications, if any.
5. Strongest non-pattern explanation of the workflow.
6. Anything that still needs to be traced before candidate verification.

Worked for 2m 27s

Inbound Travel-Email Processing — Source Investigation 02

Pattern classification pending DaPlanStan · Uploaded original source ZIP · Static analysis

1. Main finding: Parsing and creating trip records are separate operations

The source establishes three important boundaries:

1. **Email intake:** `TripMailbox` identifies the trip, checks the sender and creates an `InboxItem`.
2. **AI extraction:** `ParseInboxItemJob` sends email information to Anthropic and saves the resulting structured data in the existing `InboxItem`.
3. **Record creation:** `InboxItemsController#update` separately invokes application methods that create or link `TripItem` and `Expense` records when the user selects confirmation or relinking.

Parsing an email does not itself create itinerary items or expenses. This distinction is directly supported by `ParseInboxItemJob#perform` at lines 99–112 and `InboxItemsController#update` at lines 16–35.

An approved sender allows an email to proceed to parsing, but approval is not the same as confirmation of its extracted travel information.

2. How incoming email enters and is assigned to a trip

The application's mailbox entry point is:

`app/mailboxes/application_mailbox.rb:1–3`

Ruby

```ruby
class ApplicationMailbox < ActionMailbox::Base
  routing(all: :trip)
end
```